

# LangChain RAG Series - Class 2

## Document Loaders: The Foundation of RAG

Welcome to Class 2 of our LangChain RAG Series!

In the previous class, we explored Text Splitters - the tools that break large documents into manageable chunks. Today, we step back to understand where those documents come from in the first place.

Document Loaders are the entry point of any RAG (Retrieval-Augmented Generation) pipeline. They serve a critical purpose: converting data from various heterogeneous sources into a standardized format that the rest of your pipeline can understand.

Why Standardization Matters:

Imagine building an AI application that needs to read from PDFs, websites, CSV files, and databases. Without a standard interface, you would need to write custom parsing code for each source. Document Loaders solve this by providing a unified API - regardless of the source, the output is always a list of Document objects.

Each Document object contains two essential components:

1. Page Content - The actual text extracted from the source
2. Metadata - Contextual information like source file, page number, author, etc.

This standardization ensures that downstream components (splitters, embedders, vector stores) work seamlessly without knowing the original data source.

# Types of Document Loaders

LangChain provides hundreds of document loaders organized by category. Here are the core loaders every developer should know:

## 1. TextLoader

- Loads plain text files (.txt)
- Best for: logs, code snippets, transcripts
- Output: Single Document per file

## 2. PyPDFLoader

- Loads PDF files using PyPDF2 internally
- Best for: text-based PDFs, reports, research papers
- Output: One Document per page
- Note: Not suitable for scanned/image-based PDFs

## 3. DirectoryLoader

- Loads entire folders of files
- Best for: bulk processing, document libraries
- Supports file pattern matching (e.g., \*.pdf)
- Can specify different loaders for different file types

## 4. CSVLoader

- Loads CSV files row by row
- Best for: tabular data, datasets, analytics
- Output: One Document per row with metadata

## 5. WebBaseLoader

- Loads content from web pages
- Uses requests + BeautifulSoup internally
- Best for: static pages, blogs, documentation
- Note: JavaScript-heavy sites need SeleniumLoader

# Eager Loading vs Lazy Loading

An important design pattern in Document Loaders is the loading strategy:

Eager Loading:

- Loads ALL documents into memory at once
- Returns a complete list of Document objects
- Best for: Small datasets, quick prototyping
- Risk: High memory usage with large files

Lazy Loading:

- Returns a generator that yields one Document at a time
- Processes documents on-demand
- Best for: Large datasets, streaming, memory-constrained environments
- Benefit: Low memory footprint, faster startup

Example Scenario:

Loading a directory with 50 PDFs (2000+ pages total):

- Eager: Loads all 2000 pages into RAM before you can use any
- Lazy: Starts processing immediately, one page at a time

Most loaders in LangChain support both `.load()` and `.lazy_load()` methods, giving you flexibility based on your use case.

Custom Loaders:

If your data source doesn't have a built-in loader, LangChain allows you to create custom loaders by extending the `BaseLoader` class and implementing `.load()` and `.lazy_load()` methods.

Next Class Preview:

We will learn how to convert the chunks from Text Splitters into vector embeddings and store them in vector databases for efficient retrieval.